

Especificación de Requisitos

Sistema de Gestión de Citas Hospitalarias con Asistencia Robótica

Daniela Patricia Feversani y David Roldán

16/10/2025

1. Introducción

En las prácticas anteriores se han introducido los fundamentos de la orientación a objetos para, a continuación, aplicar estos al diseño orientado a objetos de un sistema ficticio de gestión de citas. Ahora empezaremos a implementar este diseño orientado a objetos a C++. Previamente, hemos de repasar concienzudamente el diagrama de clases de la práctica anterior, porque la implementación ha de ser una conversión de este diagrama a código

2. Visión general del sistema

Lo primero a tener en cuenta a la hora de implementar el sistema es aplicar convenientemente los especificadores de acceso a los miembros de las clases. Ya hemos visto los especificadores **public** y **private**. Los especificadores de acceso determinan la visibilidad, o accesibilidad de los atributos y operaciones de un objeto para otros objetos. Antes de empezar a implementar nuestro diseño, debemos considerar qué atributos y operaciones de nuestras clases deben ser **public** y cuales deben ser **private**.

Ya hemos visto que, por lo general, los miembros de datos deben ser **private**, y que los métodos invocados por los clientes de una clase dada deben ser **public**. Sin embargo, los métodos que se llaman sólo por otros métodos de la clase como funciones utilitarias deben ser generalmente **private**. UML emplea marcadores de visibilidad para modelar la visibilidad de los atributos y las operaciones. La visibilidad pública se indica mediante la colocación de un signo (+) antes de una operación o atributo; un signo menos (-) indica una visibilidad privada.

3. Ejercicio

Ahora ya sí, estamos listos para empezar a implementar el sistema. Primero convertiremos las clases del diagrama de clases en archivos de cabecera (.h) de C++. Este código

representará el esqueleto del sistema. (Más adelante modificaremos estos archivos para incorporar nuevos conceptos como la herencia).

Sigue estas pautas para cada clase:

1. Usa el nombre que se localiza en el primer compartimento de una clase en el diagrama de clases para definir la clase en un archivo de encabezado. Recuerda utilizar las directivas del preprocesador **#ifndef**, **#define** y **#endif** para evitar que el archivo de encabezado se incluya más de una vez en el programa.
2. Usa los atributos que se localizan en el segundo compartimento de la clase para declarar los atributos.
3. Usa las asociaciones descritas en el diagrama de clases para declarar las referencias a otros objetos. Por ejemplo, un Paciente puede estar asociado a muchas citas. Por el momento, puedes definir asociaciones de este estilo como un array. En esta clase se deberán incluir mediante sentencias **#include** los archivos de encabezado que contienen las definiciones de las otras clases que se estén usando.
4. Mejora del punto anterior. En algunos casos, algunos objetos (clases) solo referencian a un único objeto de otra clase. Si bien un paciente puede tener muchas citas, un médico solo tiene una especialidad, un robot solo tiene un sensor LIDAR y un Médico solo tiene una agenda. En este tipo de casos, estas clases contienen referencias a objetos de las otras clases, no verdaderos objetos; y la cantidad de información requerida por el compilador para crear una referencia difiere de la que se requiere para crear un objeto. Recuerda que para crear el objeto, el programador debe proporcionar al compilador una definición de la clase que introduzca el nombre de la clase como nuevo tipo definido por el usuario, y que indique los miembros de datos que determinen cuanta memoria se requiere para almacenar el objeto. Sin embargo, al declarar una referencia (un apuntador) a un objeto, sólo se requiere que el compilador sepa que la clase del objeto existe; esto es, no necesita conocer el tamaño del objeto. Cualquier referencia (o apuntador), sin importar la clase del objeto al que hace referencia, sólo contiene la dirección de memoria de dicho objeto. La cantidad de memoria requerida para almacenar una dirección es una característica física del hardware de la computadora. Por ende, el compilador conoce el tamaño de cualquier referencia. Como resultado, es innecesario incluir el archivo de encabezado completo de una clase cuando se declara sólo una referencia a un objeto de esa clase; simplemente necesitamos introducir el nombre de la clase, pero no necesitamos proporcionar la distribución de los datos del objeto, ya que el compilador conoce de antemano el tamaño de todas las referencias. C++ proporciona una instrucción conocida como declaración anticipada, la cual indica que un archivo de encabezado contiene referencias o apuntadores a una clase, pero ésta se encuentra fuera del archivo de encabezado. Así, podemos reemplazar las instrucciones **#include** de las clases mencionadas por declaraciones anticipadas de las clases con las que están asociadas. Por ejemplo, en lugar de incluir mediante **#include** el archivo de encabezado completo para cada una de las clases, sólo colocamos una declaración anticipada de

cada clase en el archivo de encabezado, por ejemplo, de la clase Robot. Ten en cuenta que si la clase Robot contara con objetos actuales en vez de referencias, entonces tendríamos que incluir los archivos de encabezado completo. Usar una declaración anticipada (donde sea posible) en vez de incluir un archivo de encabezado completo nos ayuda a evitar un problema del preprocesador conocido como inclusión circular. Este problema ocurre cuando el archivo de encabezado para la clase A incluye el archivo de encabezado para la clase B y viceversa. Algunos preprocesadores no pueden resolver esta situación, lo cuál produce un error de compilación. Por ejemplo, si la clase A sólo utiliza una referencia a un objeto de la clase B, entonces la instrucción `#include` en el archivo de encabezado de la clase A se puede reemplazar por una declaración anticipada de la clase B, para evitar la inclusión circular.

5. Usa las operaciones que se localizan en el tercer compartimento de la clase para escribir los prototipos y los métodos. Si todavía no se ha especificado un tipo de valor de retorno para una operación, declararemos el método con el tipo de valor de retorno **void**
6. Implementa el `.cpp` de cada clase, aunque los métodos no tengan contenido.

4. Entrega

Entrega un `.zip` con todos los `.h` y los `.cpp` de las clases creadas. Acuérdate de crear un `main.cpp` para comprobar si hay errores de sintáxis.